

8.1 Analyse des algorithmes

8.1.1 Définitions

Définition – Terminaison d'un algorithme

Prouver la terminaison d'un algorithme signifie montrer que cet algorithme se terminera en un temps fini. On utilise pour cela un **variant de boucle**.

Définition – Correction partielle d'un algorithme

Un algorithme est dit partiellement correct s'il est correct dès qu'il termine.

Définition – Correction totale d'un algorithme

Un algorithme est dit totalement correct s'il est partiellement correct et s'il termine.

Prouver la correction partielle d'un algorithme signifie montrer que cet algorithme fournit bien la solution au problème qu'il est sensé résoudre. On utilise pour cela un **invariant de boucle**.

Définition – Invariant de boucle

Un invariant de boucle est une propriété dépendant des variables de l'algorithme, qui est vérifiée à chaque passage dans la boucle.

Définition – Complexité

La complexité algorithmique est l'étude des ressources requises pour exécuter un algorithme, en fonction d'un paramètre (souvent, la taille des données d'entrée). Les deux ressources en général étudiées sont :

- ▶ le temps nécessaire à l'exécution de l'algorithme ;
- ▶ la mémoire nécessaire à l'exécution de l'algorithme (en plus des données d'entrée).

8.1.2 Un exemple ...

Objectif

L'objectif est ici de montrer la nécessité d'utiliser un invariant de boucle. Pour cela, on propose la fonction suivante sensée déterminer le plus petit entier n strictement positif tel que $1 + 2 + \dots + n$ dépasse strictement la valeur entière strictement positive v . Cette fonction renvoie-t-elle le bon résultat ? Desfois ? Toujours ?

Montrer intuitivement que $foo(v)$ se termine pour $v \in \mathbb{N}^*$.

8.1	Analyse des algorithmes	1
8.2	Terminaison d'un algorithme	2
8.3	Correction d'un algorithme	4
8.4	Complexité algorithmique	5
8.5	Complexité des algorithmes	6
8.6	Profiling des algorithmes	11

- ▶ Terminaison.
- ▶ Correction partielle.
- ▶ Correction totale.
- ▶ Variant. Invariant.
- ▶ Complexité.

```

1 def foo(v:int) -> int:
2     r = 0
3     n = 0
4     while r < v :
5         n = n+1
6         r = r+n
7     return n
    
```

L'algorithme se terminera si on sort de la boucle `while`. Il faut pour cela que la condition $r < v$ devienne fausse (cette condition est vraie initialement). Pour cela, il faut que r devienne supérieure ou égale à v dont la valeur ne change jamais.

n étant incrémenté de 1 à chaque itération, la valeur de r augmente donc à chaque itération. Il y aura donc un rang n au-delà duquel r sera supérieur à v . L'algorithme se termine donc.

Que renvoie `foo(9)` ? Cela répond-il au besoin ?

Début de la i ^e itération	r	n	$r < v$
Itération 1	0	0	$0 < 9 \Rightarrow \text{True}$
Itération 2	1	1	$1 < 9 \Rightarrow \text{True}$
Itération 3	3	2	$3 < 9 \Rightarrow \text{True}$
Itération 4	6	3	$6 < 9 \Rightarrow \text{True}$
Itération 5	10	4	$10 < 9 \Rightarrow \text{False}$

La fonction renvoie 4. On a $1 + 2 + 3 + 4 = 10$. On dépasse strictement la valeur 9. La fonction répond au besoin dans ce cas.

Que renvoie `foo(10)` ? Cela répond-il au besoin ?

Début de la i ^e itération	r	n	$r < v$
Itération 1	0	0	$0 < 10 \Rightarrow \text{True}$
Itération 2	1	1	$1 < 10 \Rightarrow \text{True}$
Itération 3	3	2	$3 < 10 \Rightarrow \text{True}$
Itération 4	6	3	$6 < 10 \Rightarrow \text{True}$
Itération 5	10	4	$10 < 10 \Rightarrow \text{False}$

La fonction renvoie 4. On a $1 + 2 + 3 + 4 = 10$. On ne dépasse pas strictement la valeur 10. La fonction ne répond pas au besoin dans ce cas.

Résultat –

La fonction proposée ne remplit pas le cahier des charges. Aurait-on pu le prouver formellement ?

8.2 Terminaison d'un algorithme

8.2.1 Variant de boucle

Définition – Variant de boucle

Un variant de boucle permet de prouver la terminaison d'une boucle conditionnelle. Un variant de boucle est une **quantité entière positive** à l'entrée de chaque itération de la boucle et qui **diminue strictement à chaque itération**.

Théorème –

Si une boucle admet un variant de boucle, elle termine.

Propriété –

Un algorithme qui n'utilise ni boucles inconditionnelles (boucle `for`) ni récursivité termine toujours. Ainsi, la question de la terminaison n'est à considérer que dans ces deux cas.

Reprenons l'exemple précédent.

Dans cet exemple montrons que la quantité $u_n = v - r$ est un variant de boucle :

- ▶ initialement, $r = 0$ et $v > 0$; donc $u_0 > 0$;
- ▶ à la fin de l'itération n , on suppose que $u_n = v - r > 0$ et que $u_n < u_{n-1}$;
- ▶ à l'itération $n + 1$:
 - cas 1 : $r \geq v$. Dans ce cas, n et r n'évoluent pas l'hypothèse de récurrence reste vraie. On sort de la boucle `while`. L'algorithme termine,
 - cas 2 : $r < v$. Dans ce cas, à la fin de l'itération $n + 1$, montrons que $u_{n+1} < u_n$: $u_{n+1} = v - (r + n + 1) = u_n - n - 1$ soit $u_{n+1} = u_n - n - 1$ et donc $u_{n+1} < u_n$. L'hypothèse de récurrence est donc vraie au rang $n + 1$.

Au final, $u_n = v - r$ est donc un variant de boucle et la boucle se termine.

```

1 def foo(v:int) -> int:
2     r = 0
3     n = 0
4     while r < v :
5         n = n+1
6         r = r+n
7     return n

```

8.2.2 Un second exemple ressemblant...¹

1: https://marcdefalco.github.io/pdf/complet_python.pdf

Considérons l'algorithme suivant qui, étant donné un entier naturel n strictement positif (inférieur à 2^{30}), détermine le plus petit entier k tel que $n \leq 2^k$.

```

1 def plus_grande_puissance2(n):
2     k = 0
3     p = 1
4     while p < n:
5         k = k+1
6         p = p*2
7     return k

```

[1] Dans l'exemple précédent, la quantité $n - p$ est un variant de boucle :

- ▶ au départ, $n > 0$ et $p = 1$ donc $n - p \geq 0$;
- ▶ comme il s'agit d'une différence de deux entiers, c'est un entier. Et tant que la condition de boucle est vérifiée $p < n$ donc $n - p > 0$.
- ▶ lorsqu'on passe d'une itération à la suivante, la quantité passe de $n - p$ à $n - 2p$ ou $2p - p > 0$ car $p \geq 1$. Il y a bien une stricte diminution.

[2] Montrons que, la quantité $u_j = n - p$ est un variant de boucle :

- ▶ initialement, $n > 0$ et $p = 1$ donc $n - p \geq 0$;
- ▶ à la fin de l'itération j , on suppose que $u_j = n - p > 0$ et $u_j < u_{j-1}$;
- ▶ à la fin de l'itération suivante, $u_{j+1} = n - 2p = u_j - p$. p est positif donc u_{j+1} est un entier et $u_{j+1} < u_j$. Par suite, ou bien $u_{j+1} < 0$ c'est à dire que $n - p < 0$ soit $p > n$. On sort donc de la boucle. Ou bien, $u_{j+1} > 0$, et la boucle continue.

$n - p$ est donc un variant de boucle.

8.3 Correction d'un algorithme

8.3.1 Invariant de boucle

Méthode –

Pour montrer qu'une propriété est un invariant de boucle dans une boucle `while` :

- ▶ le propriété doit être vérifiée avant d'entrer dans la boucle ;
- ▶ la propriété doit être vraie en entrée de boucle ;
- ▶ la propriété doit être vraie en fin de boucle.

Reprenons un des exemples précédents. Reconsidérons l'algorithme suivant qui, étant donné un entier naturel n strictement positif (inférieur à 2^{30}), détermine le plus petit entier k tel que $n \leq 2^k$.

```

1 def plus_grande_puissance2(n)
2   :
3   k = 0
4   p = 1
5   while p < n:
6       k = k+1
7       p = p*2
8   return k

```

Montrons que la propriété suivante est un invariant de boucle : $p = 2^k$ et $2^{k-1} < n$.

- ▶ **Initialisation** : à l'entrée dans la boucle $k = 0$ et $p = 1, n \in \mathbb{N}^*$
 - d'une part on a bien $1 = 2^0$;
 - d'autre part $2^{-1} < n$.
- ▶ On considère que la propriété est vraie au n^{e} tour de boucle c'est à dire $p = 2^k$ et $2^{k-1} < n$.
- ▶ Au tour de boucle suivant :
 - **ou bien** $p \geq n$. Dans ce cas, on sort de la boucle et on a toujours $p = 2^k$ et $2^{k-1} < n$ (propriété d'invariance). La propriété est donc vraie au tour $n + 1$.
 - **ou bien** $p < n$. Dans ce cas, il faut montrer que $p = 2^{k+1}$ et $2^k < n$. Etant entrés dans la boucle, $p < n \Rightarrow 2^k < n$. De plus, en fin de boucle, $p \rightarrow p * 2$ et $k \rightarrow k + 1$. On a donc $p \leftarrow 2^k * 2 = 2^{k+1}$.

La propriété citée est donc un invariant de boucle.

8.3.2 Un « contre exemple »

```

1 def foo(v:int) -> int:
2   assert v>0
3   r = 0
4   n = 0
5   while r <= v :
6       n = n+1
7       r = r+n
8   return n

```

Reprenons le tout premier exemple où on cherche le plus petit entier n strictement positif tel que $1 + 2 + \dots + n$ dépasse strictement la valeur entière strictement positive v .

La propriété suivante est-elle un invariant de boucle : $r = \sum_{i=0}^n i$ et $\sum_{i=0}^{n-1} i \leq v, n \in \mathbb{N}^*$?

La réponse est directement NON, car la phase d'initialisation n'est pas vérifiée car $n = 0$ et $n \notin \mathbb{N}^*$. Cela signifie donc que l'algorithme proposé ne répond pas au cahier des charges.

Modifions alors l'algorithme ainsi.

```

1 def foo2(v:int) -> int:
2   assert v>0
3   r = 1
4   n = 1
5   while r <= v :
6       n = n+1
7       r = r+n
8   return n

```

Montrons que la propriété suivante est un invariant de boucle : $r = \sum_{i=0}^n i$ et $\sum_{i=0}^{n-1} i \leq v, n \in \mathbb{N}^*$.

- ▶ **Initialisation** : à l'entrée dans la boucle $r = 1$ et $n = 1, n \in \mathbb{N}^*$
 - d'une part on a bien $r = \sum_{i=0}^1 i = 1$;
 - d'autre part $\sum_{i=0}^0 i = 0 < v$ et $v > 0$ (spécification de la fonction).

- On considère que la propriété est vraie au début du n^e tour de boucle c'est-à-dire

$$r = \sum_{i=0}^n i \text{ et } \sum_{i=0}^{n-1} i \leq v.$$

- À la fin du n^e tour de boucle, $n_{n+1} = n_n + 1$ et $r_{n+1} = r_n + n_{n+1} = r_n + n_n + 1 = \sum_{i=0}^n (i) + n_n + 1 = \sum_{i=0}^{n+1} i$ (car $n_n = n$). On a alors,
- ou bien $r_{n+1} > v$ et on sort de la boucle; on peut renvoyer n .
 - ou bien $r_{n+1} \leq v$ et donc $\sum_{i=0}^n i \leq v$.

La propriété citée est donc un invariant de boucle.

8.4 Complexité algorithmique

8.4.1 Mise en évidence du problème

Prenons l'exemple de la recherche d'un élément dans une liste :

```

1 def is_number_in_list(nb, tab):
2     """Renvoie True si le nombre nb est dans la liste de nombres tab
3     Entrées :
4         * nb, int -- nombre entier
5         * tab, list -- liste de nombres entiers
6     """
7     for i in range(len(tab)):
8         if tab[i]==nb:
9             return True
10    return False

```

Exemple –

À partir de l'algorithme précédent, évaluer :

- le nombre de tour de boucles dans le pire des cas;
- le nombre de tour de boucles dans le meilleur des cas.

```

1 def is_number_in_list_dicho(nb, tab):
2     """
3     Recherche d'un nombre par dichotomie dans un tableau trié.
4     Renvoie l'index si le nombre nb est dans la liste de nombres tab.
5     Renvoie None sinon.
6     Entrées :
7         * nb,int -- nombre entier
8         * tab,list -- liste de nombres entiers triés
9     """
10    g, d = 0, len(tab)-1
11    while g <= d:
12        m = (g + d) // 2
13        if tab[m] == nb:
14            return m
15        if tab[m] < nb:

```

```

16         g = m+1
17     else:
18         d = m-1
19     return None

```

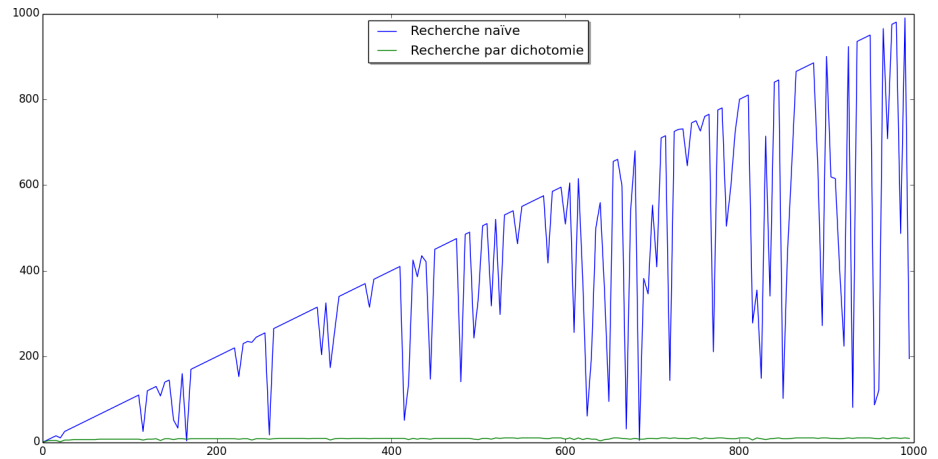


FIGURE 8.1 – Évolution du nombre d’opérations pour rechercher un nombre dans une liste de 1 à 1000 nombres

8.5 Complexité des algorithmes

8.5.1 Présentation

Il existe souvent plusieurs façons de programmer un algorithme. Si le nombre d’opérations à effectuer est peu important et les données d’entrée de l’algorithme sont de faibles tailles, le choix de la solution importe peu. En revanche, lorsque le nombre d’opérations et la taille des données d’entrée deviennent importants, deux paramètres deviennent déterminants : le temps d’exécution et l’occupation mémoire.

Définition – Complexité en temps

La complexité en temps donne le nombre d’opérations effectuées lors de l’exécution d’un programme. On appelle C_o le coût en temps d’une opération o .

Définition – Complexité en mémoire (ou en espace)

La complexité en mémoire donne le nombre d’emplacements mémoires occupés lors de l’exécution d’un programme.

Remarque

On distingue la complexité dans le pire des cas, la complexité dans le meilleur des cas, ou la complexité en moyenne. En effet, pour un même algorithme, suivant les données à manipuler, le résultat sera déterminé plus ou moins rapidement. Généralement, on s’intéresse au cas le plus défavorable à savoir, la complexité dans le pire des cas.

Définition – Complexité algorithmique – Notation $\mathcal{O}$ – [Bournez]

Soient f et g deux fonctions $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. On note $f(n) = \mathcal{O}(g(n))$ lorsqu'il existe $c \in \mathbb{R}^+$ et $n_0 \in \mathbb{N}$ tels que pour tout $n \geq n_0$,

$$f(n) \leq c \cdot g(n)$$

Intuitivement, cela signifie que f est inférieur à g à une constante multiplicative près pour les données suffisamment grandes.

Exemple –

Par ordre de complexité croissante on a :

- ▶ $\mathcal{O}(1)$: algorithme s'exécutant en temps constant, quelle que soit la taille des données ;
- ▶ $\mathcal{O}(\log(n))$: algorithme rapide (complexité logarithmique) (Exemple : recherche par dichotomie dans un tableau trié) ;
- ▶ $\mathcal{O}(n)$: algorithme linéaire ;
- ▶ $\mathcal{O}(n \cdot \log(n))$: algorithme quasi-linaire ;
- ▶ $\mathcal{O}(n^2)$: complexité quadratique ;
- ▶ $\mathcal{O}(n^3)$: complexité cubique ;
- ▶ $\mathcal{O}(2^n)$: complexité exponentielle.

8.5.2 Coût temporel d'un algorithme et d'une opération**Résultat –**

On considère que le coût élémentaire C_e correspond au coût d'une affectation, d'une comparaison ou de l'évaluation d'une opération arithmétique.

Exemple –

Chacune de ces 3 instructions ont le même coût temporel C_e . On a donc $C_{\text{foo}}(n) = 3C_e$.

`foo` prendra (a priori) toujours le même temps d'exécution. On a donc $C_{\text{foo}}(n) = \mathcal{O}(1)$. On parle de complexité constante.

```
1 def foo():
2     a=20
3     a<=100
4     a+a
```

Résultat –

Pour une séquence de deux instructions de coûts respectifs C_1 et C_2 , le coût total est de la séquence est de $C_1 + C_2$.

Exemple –

Soit la fonction ci-contre.

Le coût temporel correspond à l'addition du coût élémentaire de l'affectation ajouté au coût de l'affichage. On a donc $C_{\text{foo}}(n) = C_e + C_{\text{print}}$. Le coût du `print` est variable en fonction, par exemple, de la longueur de la chaîne à afficher.

```
1 def foo():
2     a=20
3     print(a)
```

foo prendra (a priori) toujours le même temps d'exécution. On a donc $C_{\text{foo}}(n) = \mathcal{O}(1)$. On parle de complexité constante.

Résultat –

Le coût d'un test `if test : inst_1 else : inst_2` est inférieur ou égal au maximum du coût de l'instruction 1 et du coût de l'instruction 2 additionné au coût du test (coût élémentaire).

```
1 def foo(x) :
2     if x < 0 :
3         x = x + 1
4         x = x + 2
5     else :
6         x = x + 1
```

Exemple –

Soit le programme suivant (sans application réelle) :

La comparaison a un coût élémentaire C_e . Dans le « pire » des cas, on réalise deux additions et deux affectations. Le coût temporel total est donc $C_{\text{foo}}(n) = C_e + \max(4C_e, 2C_e) = 5C_e$. foo prendra (a priori) toujours le même temps d'exécution. On a donc $C_{\text{foo}}(n) = \mathcal{O}(1)$. On parle de complexité constante.

Résultat –

Le coût d'une boucle `for i in range(n) : inst` est égal à : n fois le coût de l'instruction `inst` si elle est indépendante de la valeur de i .

```
1 def foo(n) :
2     res = 0
3     for i in range(n) :
4         res = res + i
5     return res
```

Exemple –

On peut considérer que l'incrément de i a un coût C_e . $C_{\text{foo}}(n) = C_e + n \times 3C_e = \mathcal{O}(n)$. La durée de l'algorithme croît linéairement avec la valeur de n .

Résultat –

Soit la boucle `while cond : inst`, la condition `cond` faisant intervenir un variant de boucle. Il est donc possible de connaître le nombre n d'itérations de la boucle.

Résultat –

Dans la pratique, on cherche toujours à **majorer** le coût temporel d'un algorithme. En conséquences :

- ▶ il est inutile de compter exactement le nombre d'opérations ;
- ▶ il « suffit » de se placer dans le pire des cas et de compter le nombre de fois qu'est réalisée l'opération se réalisant le plus de fois.

8.5.3 Exemple

Exemple – Calcul de factorielle

Complexité en mémoire C_M : lors de l'exécution du programme, il sera nécessaire de stocker les variables suivantes : n , res , i .

```
1 def factorielle(n) :
2     if n == 0 :
3         return 1
4     else :
5         i = 1
6         res = 1
7         while i <= n :
8             res = res * i
9             i = i + 1
10    return res
```

La complexité en mémoire est donc constante : $C_M = \mathcal{O}(1)$.

Complexité en temps C_T (1)

Compte-tenu d'un des résultats précédents, dans le pire des cas, on est dans le pire. La boucle `while` est réalisée n fois; donc on peut directement dire que $C_T(n)$ est un $\mathcal{O}(n)$.

Complexité en temps C_T (2) Une justification plus exhaustive serait la suivante.

La première comparaison a un coût élémentaire C_e .

Pour $n = 0$ le coût du retour est C_r .

Pour $n \neq 0$:

- ▶ les deux affectations ont un coût respectif C_e ;
- ▶ la boucle tant que sera réalisée n fois. Pour chaque itération,
 - la multiplication ainsi que l'affectation ont chacune un coût C_e ;
 - l'incréméntation et l'affectation ont chacune un coût C_e ;
- ▶ le coût du retour est C_r .

En conséquence, la complexité en temps s'élève à :

$$C_T(n) = C_e + \max(C_r; C_e + C_e + n(4C_e) + C_r)$$

Ainsi $C_T(n) = C_e(3 + 4n) + C_r$ et $C_T(n) \underset{+\infty}{\sim} 4C_en$ lorsque n tend vers l'infini. On parle d'une complexité algorithmique linéaire, notée $\mathcal{O}(n)$.

Il est fréquent que la complexité en temps soit améliorée au prix d'une augmentation de la complexité en espace, et vice-versa. La complexité dépend notamment :

- ▶ de la puissance de la machine sur laquelle l'algorithme est exécuté ;
- ▶ du langage et compilateur / interpréteur utilisé pour coder l'algorithme ;
- ▶ du style du programmeur.

8.5.4 D'autres exemples

Recherche d'un maximum

Exemple – Recherche d'un maximum

Soit une liste de nombre entiers désordonnés. Comment déterminer le plus grand nombre de la liste ?

Intuitivement, une solution est de parcourir la liste d'éléments et de déterminer le plus grand élément par comparaisons successives.

Dans ce cas, le coût temporel est : $C_T(n) = C_e + n(2C_e)$. Ici encore, la complexité de cet algorithme est linéaire car $C_T(n) \underset{+\infty}{\sim} 2C_en$.

Par ailleurs, en comptant le nombre de comparaisons, on observe aussi qu'il y a environ n comparaisons. On a donc $C_T(n)$ est en $\mathcal{O}(n)$.

```

1 def cherche_max(L : list) ->
2   int :
3   maxi = L[0]
4   for i in range(1, len(L)):
5     if tab[i] > maxi :
6       maxi = tab[i]
7   return maxi

```

Suite

Soit la suite u_n définie par récurrence pour tout $n \in \mathbb{N}^*$ par $\begin{cases} u_1 = 1 \\ u_{n+1} = \frac{u_n + 6}{u_n + 2} \end{cases}$.

```

1 def un_it (n) :
2   if n == 1 :
3     return 1

```

```

4     else :
5         u = 1
6         for i in range(2,n+1):
7             u = (u+6)/(u+2)
8         return u

```

La boucle `for` s'exécute n fois. Cet algorithme est en $\mathcal{O}(n)$.

```

1 def un_rec (n) :
2     if n == 1 :
3         return 1
4     else :
5         return (un_rec(n-1)+6)/(un_rec(n-1)+2)

```

A l'itération n , $C(n) = 2 \times C(n-1)$. Il s'agit donc d'une suite géométrique et $C(n) = C(0) \times 2^n$. On a donc une complexité exponentielle en $\mathcal{O}(2^n)$.

```

1 def un_rec_v2 (n):
2     if n == 1 :
3         return 1
4     else :
5         v = un_rec_v2(n-1)
6         return (v+6)/(v+2)

```

A l'itération n , $C(n) = 1 + C(n-1)$. Il s'agit donc d'une suite arithmétique et $C(n) = C(0) + n$. On a donc une complexité linéaire en $\mathcal{O}(n)$.

Diviser pour régner – recherche dichotomique

```

1 def recherche_dichotomique(x,
2     a):
3     g, d = 0, len(a)-1
4     while g <= d:
5         m = (g + d) // 2
6         if a[m] == x:
7             return m
8         elif a[m] < x:
9             g = m+1
10        else:
11            d = m-1
12    return None

```

Exemple –

On peut montrer que la suite $d - g$ décroît strictement (car d décroît et g croît). Dans ce cas, la difficulté consiste en déterminer le nombre de fois que sera exécutée la boucle `while`. On note $C_w = C_e + \max(C_e + C_r; 2C_e + 2C_r; 3C_e + C_r) = C_e + \max(C_e + C_r; 4C_e)$ le coût d'une itération de la boucle `while`.

Au cours de l'algorithme, on va devoir diviser en 2 la taille le tableau jusqu'à ce qu'on trouve (ou pas) l'élément recherché. On cherche donc combien de fois m on peut diviser par 2 la taille du tableau n :

$$\frac{n}{2^m} \geq 1 \iff n \geq 2^m \iff \ln(n) \geq m \ln(2)$$

On parlera ici de complexité logarithmique.

Résultat –

Pour une opération ayant un temps d'exécution de $10^{-9}s$, on peut calculer le temps d'exécution en fonction du nombre de données et de la complexité de l'algorithme :

Données	$\mathcal{O}(\log(n))$	$\mathcal{O}(n)$	$\mathcal{O}(n \cdot \log(n))$	$\mathcal{O}(n^2)$	$\mathcal{O}(2^n)$
100	$2 \cdot 10^{-9} s$	$0,1 \cdot 10^{-6} s$	$0,2 \cdot 10^{-6} s$	$10 \cdot 10^{-6} s$	$1,26765 \cdot 10^{21} s$
1 000	$3 \cdot 10^{-9} s$	$1 \cdot 10^{-6} s$	$3 \cdot 10^{-6} s$	$0,001 s$	$1,0715 \cdot 10^{292} s$
10 000	$4 \cdot 10^{-9} s$	$10 \cdot 10^{-6} s$	$40 \cdot 10^{-6} s$	$0,1 s$	$+\infty$

8.5.5 Trompe l'œil – coûts cachés –

```
1 def recherche(e,L):
2     return e in L
```

```
1 L1 = L.copy()
```

```
1 L.append(e)
```

Il faut être vigilant lorsqu'on manipule des tableaux avec Python. En effet, de simples instructions cachent en effet des coûts pas toujours visibles. Ainsi, l'instruction `e in L` est linéaire $\mathcal{O}(n)$ car Python doit inspecter chacun des éléments d'une liste de taille `n` pour savoir si `e` est dedans ou non.

Il en est de même pour l'instruction `copy`.

Concernant la méthode `append` on pourra faire l'hypothèse que cela s'effectue en temps constant ($\mathcal{O}(1)$). Cependant, lorsqu'on ajoute une liste Python préserve des espaces d'allocations juxtaposés. Tant que la liste ne dépasse pas la taille de l'espace alloué, l'ajout d'un élément se fait à temps constant. Quand cette taille est dépassée, la liste est déplacée à un autre endroit en mémoire où plus d'espace est nécessaire. Cette copie se fait donc en temps linéaire. Ainsi, pour `append` on parle de temps amorti constant.

8.6 Profiling des algorithmes

Afin d'évaluer la performance des algorithmes, il existe des fonctionnalités permettant de compter le temps consacré à chacune des fonctions ou à chacune des instructions utilisées dans un programme <http://docs.python.org/2/library/profile.html>.

Exemple –

Voici un exemple du crible d'Eratosthène.

```
1 def crible(n):
2     tab=[]
3     for i in range(2,n):
4         tab.append(i)
5     # Liste en comprehension tab=[x for x in range(2,n)]
6     for i in range(0,len(tab)):
7         for j in range(len(tab)-1,i,-1):
8             if (tab[j]%tab[i]==0):
9                 tab.remove(tab[j])
10    return tab
11
12 import cProfile
13 cProfile.run('crible(10000)')
```

`cProfile` renvoie alors le message suivant :

```
1 28770 function calls in 1.957 seconds
2
3 Ordered by: standard name
4
5 ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
6      1    0.000    0.000    1.957    1.957 <string>:1(<module>)
7      1    0.420    0.420    1.957    1.957 eratosthene.py:4(crible)
8      1    0.000    0.000    1.957    1.957 {built-in method exec}
9     9999    0.015    0.000    0.015    0.000 {built-in method len}
```

```

10      9998      0.016      0.000      0.016      0.000 {method 'append' of 'list'
      objects}
11      1      0.000      0.000      0.000      0.000 {method 'disable' of '_lsprof.
      Profiler' objects}
12      8769      1.505      0.000      1.505      0.000 {method 'remove' of 'list'
      objects}

```

On a alors le bilan du temps passé à effectuer chacune des opérations. Ainsi pour améliorer notablement l'algorithme, le plus intéressant serait d'optimiser la méthode `remove`.

Bibliographie

- [1] François Denis <http://pageperso.lif.univ-mrs.fr/~francois.denis/algol2/chap1.pdf>
- [2] Alain Soyeur <http://asoyeur.free.fr/>
- [3] François Morain, Cours de l'Ecole Polytechnique, <http://www.enseignement.polytechnique.fr/profs/informatique/Francois.Morain/TC/X2004/Poly/www-poly009.html>.
- [4] Renaud Kerivent et Pascal Monasse, La programmation pour ... , Cours de l'École des Ponts ParisTech - 2012/2013 <http://imagine.enpc.fr/~monasse/Info>.
- [5] Olivier Bournez, Cours INFO 561 de l'Ecole Polytechnique, Algorithmes et programmation, <http://www.enseignement.polytechnique.fr/informatique/INF561/uploads/Main/poly-good.pdf>.
- [6] Wack et Al., *L'informatique pour tous en classes préparatoires aux grandes écoles*, Editions Eyrolles.

TD 1

Calculs de complexité – Sujet

Question 1 Donner la complexité temporelle, dans le pire des cas et dans le meilleur des cas, de l'algorithme suivant.

```
1 def f(L):
2     x = 0
3     for i in range(len(L)) :
4         for j in range(len(L[0])) :
5             x = x+1
6     return x
```

Question 2 Donner la complexité temporelle, dans le pire des cas et dans le meilleur des cas, de l'algorithme suivant.

```
1 def f2(n) :
2     x = 0
3     for i in range (n) :
4         for j in range (i) :
5             x += 1
6     return x
```

Question 3 Donner la complexité temporelle, dans le pire des cas et dans le meilleur des cas, de l'algorithme suivant.

```
1 def f3(n) :
2     x, i = 0 , n
3     while i > 1:
4         x += 1
5         i //= 2
6     return x
```

Question 4 Donner la complexité temporelle, dans le pire des cas et dans le meilleur des cas, de l'algorithme suivant.

```
1 def f4(n) :
2     x, i = 0 , n
3     while i > 1:
4         for j in range (n) :
5             x += 1
6         i //= 2
7     return x
```

Question 5 Donner la complexité temporelle, dans le pire des cas et dans le meilleur des cas, de l'algorithme suivant.

```
1 f5(n) :
2     x, i = 0 , n
3     while i > 1:
4         for j in range (i) :
5             x += 1
6         i //= 2
7     return x
```

Question 6 Donner la complexité temporelle, dans le pire des cas et dans le meilleur des cas, de l'algorithme suivant.

```
1 def f6(n) :  
2     x = 0  
3     for i in range (n) :  
4         j = 0  
5         while j * j < i :  
6             x += 1  
7             j += 1  
8     return x
```

TD 2

Correction des algorithmes – Sujet

Exercice

Soit la fonction de signature `rechercheMaxi(L:[int]) -> int`.

Question 1 Proposer un variant pour cet algorithme. Montrer que la boucle termine.

Question 2 Proposer un invariant pour cet algorithme. Montrer que cet algorithme est correct.

```
1 def rechercheMaxi(L) :  
2     n = len(L)  
3     max = L[0]  
4     i = 1  
5     while i < n :  
6         if L[i] > max :  
7             max = L[i]  
8         i = i+1  
9     return max
```

Exercice

Soit la fonction de signature `x_in_L(x:int,L:[int]) -> bool`.

Question 3 Proposer un variant pour cet algorithme. Montrer que la boucle termine.

Question 4 Proposer un invariant pour cet algorithme. Montrer que cet algorithme est correct.

```
1 def x_in_L(x,L) :  
2     n = len(L)  
3     i = 0  
4     while i < n :  
5         if L[i] == x :  
6             return True  
7         i = i+1  
8     return False
```

Exercice

On considère la fonction `mystere` suivante, qui étant donnés deux entiers x et y , renvoie un autre entier.

```
1 def mystere(x :int, y :int) -> int :  
2     a=0  
3     while y>0 :  
4         if y%2==1 :  
5             a=a+x  
6         x=x+x  
7         y=y//2  
8     return(a)
```

Question 5 Recopier et compléter les tableaux suivants donnant l'évolution des variables x , y et a ainsi que de la quantité $a+x*y$ lors des appels $f(7, 20)$ et $f(3, 85)$.

Question 6 Montrer que la fonction `mystere` termine.

Question 7 Conjecturer la valeur de `mystere(x, y)` et démontrer cette conjecture à l'aide d'un invariant de boucle bien choisi (on pourra appeler x_i , y_i et a_i les valeurs respectives de x , y et a à l'entrée du i^{e} tour de boucle).

Exercice

Soit la fonction suivante où L est une fonction triée.

```

1 def NombreDistinctsTri(L) :
2     n=len(L)
3     nombre=1
4     for i in range(n-1) :
5         if L[i]<L[i+1] :
6             nombre=nombre+1
7     return(nombre)

```

Question 8 Donner la signature de cette fonction.

Question 9 Donner au moins une assertion permettant de valider que les entrées sont conformes aux attendus du concepteur.

Question 10 Donner au moins un test permettant de valider la fonction.

Question 11 Montrer que cette fonction termine.

Question 12 Définir un invariant de la boucle constituant la fonction `NombreDistinctsTri(L)` et justifier la correction de cette fonction.

Exercice

Soit la fonction suivante réalisant un tri dit par insertion d'une liste.

```

1 def tri_par_selection(T):
2     """trie le tableau T dans l'ordre croissant"""
3     for i in range(len(T)):
4         ind_min = i
5         for j in range(i+1, len(T)):
6             if T[j] < T[ind_min]:
7                 ind_min = j
8         T[i],T[ind_min] = T[ind_min],T[i]

```

Question 13 Montrer que la propriété suivante est un invariant de boucle : au début de l'itération i , $T[0:i-1]$ est trié et chacun des éléments de $T[0:i-1]$ est inférieur ou égal aux éléments de $T[i:]$.

Exercice

Question 14 Montrer la terminaison et la correction de l'algorithme suivant.

```

1 def puiss(x, n):
2     if n == 0:
3         return 1
4     else :
5         return x*puiss(x,n-1)

```